DATA SCIENCE & DATA ANALYTICS USING PYTHON

TASK 2 — ADVANCED EXPANDED EXPLANATION

Business Intelligence Sales Analytics Dashboard

A practical companion to the Task 2 internship brief, expanding the SQL, PostgreSQL, Pandas, data wrangling, feature engineering, visualization, BI and Streamlit sections with worked examples, validation checks and interview preparation.

Important: The included dataset is synthetic practice data. Dashboard metrics demonstrate code behavior and are not real company results.

1. Map the deliverables to the assignment

The uploaded Task 2 brief describes an intermediate task, estimated at 40–45 days, with SQL, PostgreSQL, advanced Pandas, data wrangling, feature engineering, Plotly visualization, BI foundations and Streamlit. The final mini project is a Business Intelligence Sales Analytics Dashboard. The brief's deliverables include Python source, SQL scripts, a PostgreSQL database, dashboard, notebook, business report, GitHub repository, README, screenshots and documentation.

This project package supplies code and setup material for those deliverables. PostgreSQL is not automatically installed or populated on your computer by downloading the ZIP. Database execution and dashboard testing must be performed in your local environment.

2. End-to-end architecture

```
Sales CSV / PostgreSQL
        |
        v
Extract data (Pandas / SQL query)
        |
        v
Validate schema, types, nulls, duplicates
        |
        v
Transform: normalize fields, derive revenue/date features
        |
        v
Aggregate: KPIs, product, region, customer and month summaries
        |
        v
Visualize: Plotly charts
        |
        v
Present: Streamlit filters, metrics, tables, CSV export
        |
        v
Report, README, GitHub and screenshots
```

ETL means Extract, Transform and Load. Extract reads from a file or database; Transform validates and reshapes the data; Load saves it to a destination such as a processed CSV or database table. The provided etl.py writes a processed CSV and, if DATABASE_URL is configured, loads a staging table.

3. Dataset dictionary and data grain

A data dictionary explains what each column means. The grain is what one row represents. This sample treats one row as a sales line; therefore the mean row revenue is not necessarily average order value when one order can contain multiple lines.

Column	Meaning	Validation / notes
order_id	Order identifier	May repeat if an order has multiple lines in real data.
order_date	Date of sale	Parse as a date; reject or investigate unparseable dates.
customer_id	Anonymized customer key	Distinct customer counts depend on selected filters.
region	Sales region	Missing values become Unknown in this demonstration.
product / category	Product and its category	Validate product-to-category mapping.
quantity	Units sold	Must be numeric and positive for this example.
unit_price	Price per unit	Must be numeric and non-negative.
discount	Discount fraction	0.10 means 10%; expected range 0–1.
gross_sales	Quantity × unit price	Calculated feature.
revenue	Gross sales × (1 – discount)	Discounted sales; not profit.
year / month / day_of_week	Calendar features	Derived from order_date.

4. SQL fundamentals: how to read and write queries

SELECT, WHERE, ORDER BY and LIMIT

```
SELECT order_id, order_date, product, revenue
FROM sales_dashboard_view
WHERE order_date >= DATE '2025-01-01'
ORDER BY revenue DESC
LIMIT 10;
```

SELECT chooses columns; FROM identifies the source; WHERE filters individual rows; ORDER BY sorts results; LIMIT restricts the returned row count. In real applications, validate the date range and the table/view exists in the selected database.

Aggregation: GROUP BY and HAVING

```
SELECT region,
```

```
       COUNT(*) AS sales_lines,
       SUM(revenue) AS total_revenue,
       AVG(revenue) AS average_line_revenue
FROM sales_dashboard_view
GROUP BY region
HAVING SUM(revenue) > 100000
ORDER BY total_revenue DESC;
```

GROUP BY creates groups, then aggregate functions summarize them. WHERE filters records before aggregation; HAVING filters groups after aggregation. COUNT(*) counts rows, while COUNT(DISTINCT customer_id) counts distinct customers.

JOINs and join validation

```
SELECT s.order_id, p.product_name, s.quantity, s.revenue
FROM sales s
INNER JOIN products p ON p.product_id = s.product_id;

SELECT s.order_id, c.customer_segment
FROM sales s
LEFT JOIN customers c ON c.customer_id = s.customer_id;
```

INNER JOIN keeps matching keys only. LEFT JOIN preserves all left-side records even when the right-side key is missing. Before joining, check whether the lookup key is unique. If it is not unique, one source row may become multiple output rows and inflate revenue.

CTEs, subqueries, window functions and views

```
WITH product_totals AS (
  SELECT product, SUM(revenue) AS total_revenue
  FROM sales_dashboard_view
  GROUP BY product
)
SELECT product, total_revenue,
       DENSE_RANK() OVER (ORDER BY total_revenue DESC) AS revenue_rank
FROM product_totals;

CREATE OR REPLACE VIEW sales_dashboard_view AS
SELECT order_id, order_date, customer_id, product, region,
       quantity, unit_price, discount, revenue
FROM sales;
```

A CTE names an intermediate result. A subquery nests one query inside another. Window functions calculate rankings or running metrics without collapsing all rows. A view stores a named query definition, not usually a separate copy of its rows.

5. PostgreSQL: schema, constraints, indexes and operations

PostgreSQL is the database server. pgAdmin is a graphical tool for creating databases, running SQL and managing objects. The included schema.sql illustrates customers, products and sales tables, primary/foreign keys, CHECK constraints and indexes. The analytics SQL assumes the database has been loaded with compatible records.

• Primary key: unique row identity.

• Foreign key: enforces a relationship to another table.

• NOT NULL: value is required.

• UNIQUE: duplicate values are disallowed for the constrained column/key.

• CHECK: value must satisfy a rule, such as discount between 0 and 1.

• Index: can speed up filters/joins but consumes storage and adds write overhead.

Backup/restore can be performed with pgAdmin or PostgreSQL utilities such as pg_dump and pg_restore. The precise commands depend on the host, database name and chosen backup format. Never use a sample password in production or commit real credentials.

6. Python database connectivity and safe credentials

```
import os
import pandas as pd
from sqlalchemy import create_engine, text

engine = create_engine(os.environ["DATABASE_URL"])
with engine.connect() as conn:
    result = pd.read_sql(
        text("SELECT * FROM sales_dashboard_view WHERE region = :region"),
        conn,
        params={"region": "East"}
    )
```

SQLAlchemy manages the engine and connections; Psycopg is the PostgreSQL driver. Use parameter binding for values rather than string concatenation. Put credentials in environment variables or a secrets manager. A .env.example is a template only; it is not a working database credential.

7. Advanced Pandas techniques

Merge, join and concatenate

```
# Append datasets with the same columns
all_regions = pd.concat([north_df, south_df], ignore_index=True)

# Add customer attributes using a key
sales_with_customers = sales_df.merge(
    customers_df, on="customer_id", how="left", validate="many_to_one"
)

# Index-based alignment
combined = sales_df.join(customer_notes, how="left")
```

concat stacks data along an axis. merge combines using key columns. join often aligns on an index. The validate option can catch unexpected key cardinality and prevent silent row multiplication.

Pivot tables, crosstab and MultiIndex

```
pivot = pd.pivot_table(
    df, values="revenue", index="region", columns="category",
    aggfunc="sum", fill_value=0
)
counts = pd.crosstab(df["region"], df["category"])
multi = df.groupby(["year", "region"])["revenue"].sum()
```

Pivot tables summarize a numeric measure across dimensions. Crosstab counts category combinations by default. A MultiIndex uses more than one label level, useful for hierarchical summaries such as year and region.

Apply, lambda and rolling windows

```
df["large_sale"] = df["revenue"].apply(lambda x: x >= 10000)
monthly = df.groupby(pd.Grouper(key="order_date", freq="MS"))["revenue"].sum()
three_month_average = monthly.rolling(3, min_periods=1).mean()
```

apply runs a function over data; lambda creates a short anonymous function. Rolling windows calculate local metrics, such as a three-month moving average. Prefer vectorized expressions for common calculations because they are often clearer and faster.

8. Data wrangling and quality controls

Data wrangling transforms raw input into a reliable analysis dataset. The sample pipeline normalizes column names, parses dates, converts numeric columns, removes exact duplicates, excludes rows with missing/non-positive quantity or invalid price, fills missing region with Unknown, fills missing discount with zero and clips discount to [0,1].

```
df.isna().sum()              # missing values by column
df.duplicated().sum()        # duplicate rows
df["order_date"] = pd.to_datetime(df["order_date"], errors="coerce")
df["quantity"] = pd.to_numeric(df["quantity"], errors="coerce")
```

These are explicit demonstration assumptions, not universal policies. A real analyst should ask the data owner how to handle missing discounts, invalid dates, cancelled orders, returns and negative quantities. Keep raw data unchanged and document every exclusion or imputation.

9. Feature engineering and feature scaling

Feature engineering creates useful derived columns. In this project gross_sales, revenue, year, month, day_of_week, order_month_num and discount_band are derived from source columns.

```
df["gross_sales"] = df["quantity"] * df["unit_price"]
df["revenue"] = df["gross_sales"] * (1 - df["discount"])
df["year"] = df["order_date"].dt.year
df["month"] = df["order_date"].dt.to_period("M").astype(str)
df["day_of_week"] = df["order_date"].dt.day_name()
```

One-hot encoding creates 0/1 columns for categories. Ordinal encoding assigns ordered numeric codes only when the categories have a meaningful order. Standardization transforms x to (x – mean) / standard deviation; min-max scaling maps values to a chosen range such as 0–1. Scaling matters for many machine-learning algorithms but is not necessary for simple revenue sums.

10. Business Intelligence: KPI definitions

• Gross sales = quantity × unit price, before discounts.

• Revenue = gross sales × (1 – discount), after the discount in this example.

• Unique orders = count of distinct order_id values in the filtered data.

• Customers = count of distinct customer_id values in the filtered data.

• Average sales-line revenue = mean revenue per row; not necessarily average order value.

• Revenue share = category revenue divided by total revenue, multiplied by 100.

KPIs should be tied to a decision. For example, revenue by product helps prioritize investigation, but a high-revenue product may have low margin or high return rates. Profitability requires cost data; inventory decisions require stock and lead-time data.

11. Plotly and interactive charts

Plotly Express creates common charts quickly; Plotly Graph Objects offers more detailed control. Hover information, zoom, legends and selection help users inspect values. This project uses line charts for monthly trends and bar charts for product/region comparisons.

```
import plotly.express as px
monthly = df.groupby("month", as_index=False)["revenue"].sum()
fig = px.line(monthly, x="month", y="revenue", markers=True,
              title="Monthly Revenue Trend",
              labels={"month":"Month","revenue":"Revenue (■)"})
fig.show()
```

Chart selection should match the question. Use a line for time trends, bars for category comparisons, histograms for distributions, scatter plots for relationships and box plots for spread/outlier investigation. Avoid implying that correlation proves causation.

12. Streamlit dashboard: code walkthrough

• set_page_config defines page title and layout.

• read_csv loads cleaned data if available, otherwise raw data.

• Sidebar multiselect widgets let users choose regions, products and years.

• A Boolean mask filters rows to the selected values.

• metric cards display total revenue, gross sales, order count, customer count and mean line revenue.

• Plotly charts show monthly, product, regional and category performance.

• dataframe displays the selected records; download_button exports the filtered records as CSV.

```
streamlit run dashboard/app.py
```

Run the command from the project root. PostgreSQL is optional for the CSV demonstration. To make the app database-backed, replace the CSV loading step with a SQLAlchemy query and configure DATABASE_URL securely. Public deployment requires authentication, access control and secrets management if data is sensitive.

13. Reading the generated outputs

datasets/processed/sales_clean.csv contains the cleaned and engineered rows. reports/summary_metrics.json contains key counts and rankings. reports/eda_report.md summarizes data quality, KPIs, product/region rankings and monthly totals. reports/figures contains product, region, month and correlation charts.

The synthetic sample has 600 generated rows plus three appended duplicate rows; cleaning excludes duplicates and invalid quantities. The resulting metrics are designed to exercise the pipeline, not to support claims about an actual retailer.

14. Common errors and fixes

• ModuleNotFoundError: activate the intended virtual environment and run pip install -r requirements.txt.

• Streamlit command not found: ensure the environment is active and Streamlit installed; alternatively run python -m streamlit run dashboard/app.py.

• File not found: run commands from the project root and confirm the folder structure.

• PostgreSQL connection refused: confirm the server is running and host/port/database are correct.

• Password authentication failed: verify credentials and database permissions; do not publish secrets.

• Undefined table: run schema.sql in the intended database and load compatible data before running analytics queries.

• Unexpectedly inflated totals after a merge: check duplicate keys and use merge(validate=...).

• Empty chart after filtering: select at least one region, product and year.

• Dates or numbers become NaN: inspect source values that failed conversion before dropping rows.

15. GitHub publishing and commit

```
git init
git add .
git commit -m "Complete Task 2 Sales BI Dashboard"
```

```
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/sales-bi-dashboard-task2.git
git push -u origin main
```

Replace YOUR-USERNAME with your actual GitHub username and use the repository URL you created. Include a README with project purpose, setup, database schema, KPI definitions, screenshots, data limitations and future work. Do not commit .env files, passwords or private data.

16. Interview questions and sample answers

Why use SQL and Pandas together?

SQL retrieves and aggregates data close to the database, while Pandas supports flexible Python-based cleaning, feature engineering and exploratory analysis. The best split depends on data size, governance and performance.

What is the difference between WHERE and HAVING?

WHERE filters rows before grouping; HAVING filters grouped results after aggregation.

What is a window function?

A window function calculates a ranking or statistic across related rows without reducing the output to one row per group.

Why are foreign keys useful?

They enforce valid references between related tables, helping prevent sales records from pointing to nonexistent customers or products.

What is feature engineering?

It derives useful variables from raw fields, such as revenue after discount, month, weekday or discount band.

What are the limitations of your dashboard?

The data is synthetic, revenue excludes cost and returns, and the sample is not a live company database. PostgreSQL integration requires local setup and compatible data loading.